

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Mock Questions Set-3

Total Questions : 60

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Full Length Mock Exam Set-3(60 Questions)

Questions Question - 1

Constructor declarations can be include the access modifiers?

- 1.public
- 2.protected
- 3.private
- 4.All of the above

Explanation :

D is the correct answer.

constructor declarations may include the access modifiers public, protected, or private

Question - 2

If you run the code below, what gets printed out?

```
public class Test {  
  
    public static void main(String argv[]){  
        String s=new String("Bicycle");  
        int iBegin=1;  
        char iEnd=3;  
        System.out.println(s.substring(iBegin,iEnd));  
    }  
}
```

- 1.Bic
- 2.icy
- 3.ic
- 4.error: no method matching substring(int,char)

Explanation :

C is the correct answer.

char iEnd=3; will convert to ASCII value 3. if you declare char iEnd='a'; then it converts to 97 (ASCII value of 'a' is 97)

Question - 3

True or False.

Using the instanceof operator on an interface will cause a runtime exception.

- 1.True
- 2.False
- 3.None of the above
- 4.None of the above

Explanation :

B is the correct answer.

The instanceof operator can be used as follows: - on classes to see if a particular class is derived from another (ie, is an instance of) - on interfaces to see if a class implements a particular interface

Question - 4

Which of the following pieces of code compiles without any errors?

- 1.StringBuffer sb1 = "abcd";
- 2.Boolean b = new Boolean("abcd");
- 3.byte b = 255;
- 4.float fl = 1.2;

Explanation :

B is the correct answer.

Answer 1 is not a valid way to construct a StringBuffer. Answer 2 is a valid construction of a Boolean (any string other than "true" or "false" will result in a Boolean with a value of false) Answer 3 will fail to compile because the valid range for a byte is -128 to +127 (ie, 8 bits, signed).

Question - 5

True or False.

The StringBuffer class does not have a concat() method.

- 1.True
- 2.False
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

Refer to the JDK API documentation.

Question - 6

Which of the following methods are defined in the Object class?

- 1.toString()
- 2.equals(Object o)
- 3.wait()
- 4.All of the above

Explanation :

D is the correct answer.

Refer to the JDK API documentation.

Question - 7

True or False.

The StringBuffer class inherits from the String class.

- 1.True
- 2.False
- 3.None of the above
- 4.None of the above

Explanation :

B is the correct answer.

Refer to JDK documentation.

Question - 8

```
private class B {  
  
    public static void main(String[] args){  
        System.out.println("DD");  
        B b = new B();  
    }  
  
}
```

What is the output ?

- 1.DD
- 2.Compile Error.
- 3.Runtime Exception
- 4.None of the above.

Explanation :

B is the correct answer.

Only public, abstract and final is permitted for class modifier.

Question - 9

```
public class Point {  
    int x = 1, y = 1;  
    abstract void alert();  
}
```

Is the code compile without error ?

- 1.compile without error
- 2.compile with error , because class should be abstract.
- 3.Can't say
- 4.None of the above.

Explanation :

B is the correct answer.

If there is any abstract method in a class then the class should be abstract.

Question - 10

```
abstract class Point {  
    int x = 1, y = 1;  
    abstract void alert();  
}  
  
public class A{  
    public static void main(String[] args){  
        Point p = new Point();  
    }  
}
```

What is the output ?

- 1.compile without error
- 2.compile with error.
- 3.Can't say
- 4.None of the above.

Explanation :

B is the correct answer.

abstract class can't be instantiated.

Question - 11

Which of the below statement is true?

- 1.A subclass of an abstract class that is not itself abstract may be instantiated.
- 2.abstract class can't be instantiated.
- 3.abstract class can be instantiated.
- 4.None of the above

Explanation :

A and B is the correct answer.

A subclass of an abstract class that is not itself abstract may be instantiated.

Question - 12

Which of the following lines will print false?

```
1.  public class Test
2.  {
3.      static String s1 = "I am unique!";
4.      public static void main(String args[])
5.      {
6.          String s2 = "I am unique!";
7.          String s3 = new String(s1);
8.          System.out.println(s1 == s2);
9.          System.out.println(s1.equals(s2));
10.         System.out.println(s3 == s1);
11.         System.out.println(s3.equals(s1));
12.         System.out.println(TestClass.s4 == s1);
13.     }
14. }
15.
16. class TestClass
17. {
18.     static String s4 = "I am unique!";
19. }
```

- 1.Lines 10 and 12
- 2.Line 12 only
- 3.Line 8 and 10
- 4.None of these

Explanation :

D is the correct answer.

s3 == s1 return false but others are returns true.

Question - 13

What is the correct ordering for the import, class and package declarations when found in a Java class?

- 1.package, import, class
- 2.class, import, package
- 3.import, package, class
- 4.package, class, import

Explanation :

A is the correct answer.

Example : package com; import java.io.IOException; public class Test { }

Question - 14

What will be the result of compiling the following code:

```
public class Test {  
    static int age;  
    public static void main (String args []) {  
        age = age + 1;  
        System.out.println("The age is " + age);  
    }  
}
```

- 1.The code does not compile.
- 2.The code compiles cleanly and print 1.
- 3.The code compiles cleanly and print 0.
- 4.The code throws an Exception at Runtime.

Explanation :

B is the correct answer.

static variables are initialized automatically. default initialization value for int is 0 and boolean is false.

Question - 15

Which of the following is correct?

- 1.String temp [] = new String {"j" "a" "z"};
- 2.String temp [] = { "j " " b" "c"};
- 3.String temp = {"a", "b", "c"};
- 4.String temp [] = {"a", "b", "c"};

Explanation :

D is the correct answer.

String temp [] = {"a", "b", "c"}; where temp[] is String array.

Question - 16

What is the correct declaration of an abstract method that is intended to be public?

- 1.public abstract void add();
- 2.public abstract void add() {}
- 3.public abstract add();
- 4.public virtual add();

Explanation :

A is the correct answer.

abstract method should not have body.

Question - 17

Under what situations do you obtain a default constructor?

- 1.When you define any class
- 2.When the class has no other constructors
- 3.When you define at least one constructor
- 4.None of the above

Explanation :

B is the correct answer.

When a class has no other constructors, default will be automatically there.

Question - 18

Which of the following can be used to define a constructor for this class, given the following code:

```
public class Test {
```

```
    ...  
}
```

1. public void Test() {...}
2. public Test() {...}
3. public static Test() {...}
4. public static void Test() {...}

Explanation :

A is the correct answer.

Constructor should not have any return type and should not be static.

Question - 19

Assuming a method contains code which may raise an **Exception** (but not a **RuntimeException**), what is the correct way for a method to indicate that it expects the caller to handle that exception?

1. throw Exception
2. throws Exception
3. new Exception
4. Don't need to specify anything

Explanation :

B is the correct answer.

throws Exception from method and the caller method should catch to handle the exception.

Question - 20

Which of the following is a legal **return** type of a method overloading the following method ?

```
public void add(int a) {...}
```

1. void
2. int
3. String
4. Can be anything

Explanation :

D is the correct answer.

method overloading don't care about return type.

Question - 21

Which of the following statements is correct for a method which is overriding the following method ?

```
public void add(int a) {...}
```

- 1.the overriding method must return void
- 2.the overriding method must return int
- 3.the overriding method may return anything
- 4.None of the above

Explanation :

A is the correct answer.

In case of overriding , super class method and the sub class method should have same return type. if super class method return ArrayList then subclass overriding method should return same or sub type of ArrayList. An overriding method can also return a subtype of the type returned by the overridden method. This is called a covariant return type.

Question - 22

What is the value of y?

```
int y = 2 % 4;
```

- 1.0
- 2.2
- 3.4
- 4.Compile Error

Explanation :

B is the correct answer.

$x \% y$ is x if x is less than y.

Question - 23

What is the output for the below code ?

```
public class Test{

    public static void main(String argv[])
    {
        Test t = new Test();
        t.aMethod();
    }
}
```

```

        void aMethod() {
            static int b = 10;
            System.out.println(b);
        }
    }

```

- 1.10
- 2.Compile with error
- 3.0
- 4.Compile successfully but Runtime Exception

Explanation :

B is the correct answer.

Local variables cannot be declared static.

Question - 24

```

class A {
    B b = new B();
    C c = (C) b;
}

```

Referring to the above, when is the cast of "b" to class C allowed?

- 1.B and C are subclasses of the same superclass.
- 2.If B and C are superclasses of the same subclass
- 3.B is a subclass of C.
- 4.B is a superclass of C.

Explanation :

C is the correct answer.

For Example. B is superclass and C is subclass the B b = new (B)C id allowed.

Question - 25

Is this legal

```

public class Test {
    static { int a = 5; }
    public static void main(String[] args){
        System.out.println(a);
    }
}

```

- 1.Yes
- 2.No
- 3.Can't Say
- 4.None

Explanation :

B is the correct answer.

A variable declared in a static initialiser is not accessible outside its enclosing block.

Question - 26

Which variables can an inner class access from the class which encapsulates it?

- 1.All static variables.
- 2.All final variables.
- 3.All instance variables
- 4.All of the above

Explanation :

A is the correct answer.

inner class can access static , final and instance variables of outer class.

Question - 27

What is the output ?

```
public class Test {  
    final int k;  
    public static void main(String[] args) {  
        System.out.println(k+1);  
    }  
}
```

- 1.0
- 2.1
- 3.Compile Error.
- 4.None of the above.

Explanation :

C is the correct answer.

final variables should be initialized.

Question - 28

Inner class can extend _____ .

- 1.The top level class

- 2.The Object class
- 3.Any class or interface
- 4.It must extend an interface

Explanation :

C is the correct answer.

inner class can extends any class or interface.

Question - 29

Which one of the following is a limitation of subclassing the Thread class?

- 1.You must catch the ThreadDeath exception.
- 2.You must implement the Threadable interface.
- 3.You cannot have any static methods in the class
- 4.You cannot subclass any other class.

Explanation :

D is the correct answer.

Java don't support multiple inheritance.Subclassing the Thread Class , you can't extend any other class.

Question - 30

What is the effect of issuing a wait() method on an object ?

- 1.If a notify() method has already been sent to that object then it has no effect
- 2.The object issuing the call to wait() will halt until another object sends a notify() or notifyAll() method
- 3.An exception will be raised
- 4.The object issuing the call to wait() will be automatically synchronized with any other objects using the receiving object.

Explanation :

B is the correct answer.

issuing the call to wait() will halt until another object sends a notify() or notifyAll() method.

Question - 31

What is the effect of adding the sixth element to a vector created in the following manner?

```
new Vector(5, 10);
```

1. An IndexOutOfBoundsException exception is raised.
2. The vector grows in size to a capacity of 10 elements
3. The vector grows in size to a capacity of 15 elements
4. Nothing, the vector will have grown when the fifth element was added

Explanation :

C is the correct answer.

Constructor of Vector public Vector(int initialCapacity, int capacityIncrement).

Adding the 6th element to a vector, Size will be initialCapacity+capacityIncrement, i.e $5+10 = 15$

Question - 32

Observe the code below

```
public class Test {  
    int a[] = new int[4];  
    void aMethod()  
    {  
        int b = 0 , index;  
        a[a[b]] = a[b] = b = 2;  
        index = ??? ;  
        System.out.println(a[index]);  
    }  
}
```

What value assigned to the variable index will print a non zero result?

1. 0
2. 1
3. 2
4. 3

Explanation :

A is the correct answer.

the operands are evaluated from left to right here so in the expression `a[a[b]]` the value of `b` is 0 and `a[0]` also holds the value 0 when the expression is evaluated so the array element at index 0 is assigned the value 2.

Question - 33

What is the output of the bellow code?

```
Object a = "hello";  
String b = "hello";  
if(a == b)
```

```
System.out.println("equal");  
else  
System.out.println("not equal");
```

- 1.equal
- 2.not equal
- 3.Code not compile
- 4.None of the above

Explanation :

A is the correct answer.

It does not matter if the object reference is of type Object this will still return true because both of them point to the same String object in the string pool.

Question - 34

Is the bellow statement True or False?

The garbage collector is required to makes sure that all objects held by soft references are garbage collected before the VM throws an OutOfMemoryError.

- 1.True
- 2.False
- 3.Sometime true , sometime false.
- 4.None of the above.

Explanation :

A is the correct answer.

If the VM finds itself unable to allocate memory for a new object it is required to kick start the garbage collector and reclaim all objects that are not held by strong references before throwing an OutOfMemoryError.

Question - 35

```
public class A{  
    private void test1(){  
  
        System.out.println("test1 A");  
    }  
}
```

```
public class B extends A{  
    public void test1(){  
  
        System.out.println("test1 B");  
    }  
}
```

```

}

public class outputtest{
    public static void main(String[] args){
        A b = new B();
        b.test1();
    }
}

```

What is the output?

- 1.test1 A
- 2.test1 B
- 3.Not compile because test1() method in class A is not visible.
- 4.None of the above.

Explanation :

C is the correct answer.

Not compile because test1() method in class A is not private so it is not visible.

Question - 36

```

public class A{
    private void test1(){

        System.out.println("test1 A");
    }
}

public class B extends A{
    public void test1(){

        System.out.println("test1 B");
    }
}

public class Test{
    public static void main(String[] args){
        B b = new B();
        b.test1();
    }
}

```

What is the output?

- 1.test1 B
- 2.test1 A
- 3.Not compile because test1() method in class A is not visible
- 4.None of the above

Explanation :

A is the correct answer.

This is not related to superclass , B class object calls its own method so it compile and output normally.

Question - 37

```
public class A{
    public void test1(){

        System.out.println("test1 A");
    }
}

public class B extends A{
    public void test1(){

        System.out.println("test1 B");
    }
}

public class Test{
    public static void main(String[] args){
        A b = new B();
        b.test1();

    }
}
```

What is the output?

- 1.test1 B
- 2.test1 A
- 3.Not compile because test1() method in class A is not visible
- 4.None of the above

Explanation :

A is the correct answer.

Superclass reference of subclass point to subclass method and superclass variables.

Question - 38

```
class c1
{
    public static void main(String a[])
    {
        c1 ob1=new c1();
        Object ob2=ob1;
        System.out.println(ob2 instanceof Object);
    }
}
```

```

System.out.println(ob2 instanceof c1);
}
}

```

1. Prints true, false
2. Print false, true
3. Prints true, true
4. compile time error

Explanation :

C is the correct answer.

instanceof operator checks for instance related to class or not.

Question - 39

```

class C{
static int f1(int i) {
System.out.print(i + ",");
return 0;
}
public static void main (String[] args) {
int i = 0;
i = i++ + f1(i);
System.out.print(i);
}
}

```

1. Prints: 0,0
2. Prints: 1,0
3. Prints: 0,1
4. Compile-time error

Explanation :

B is the correct answer.

trace it by yourself.

Question - 40

```

public class C {

    public C(){

    }

}

public class D extends C {

    public D(int i){

```

```

        System.out.println("tt");
    }

    public D(int i, int j){

        System.out.println("tt");
    }

}

```

Is C c = new D(); works ?

- 1.It compile without any error
- 2.It compile with error
- 3.Can't say
- 4.None of the above

Explanation :

B is the correct answer.

C c = new D(); NOT COMPILE because D don't have default constructor. If super class has different constructor other then default then in the sub class you can't use default constructor

Question - 41

```

public class C {

    public C(){

    }

}

public class D extends C {

    public D(int i){

        System.out.println("tt");
    }

    public D(int i, int j){

        System.out.println("tt");
    }

}

```

Is C c = new D(8); works ?

- 1.It compile without any error
- 2.It compile with error
- 3.Can't say

4. None of the above

Explanation :

A is the correct answer.

C c = new D(8); COMPILE because D don't have default constructor. If super class has different constructor other than default then in the sub class you can't use default constructor

Question - 42

```
public class A {  
  
    { System.out.println("block"); }  
    public A() {  
        System.out.println("A");  
    }  
  
    public static void main(String[] args) {  
        A a = new A();  
    }  
    What will be output ?
```

1. A block
2. block A
3. A
4. None of the above

Explanation :

B is the correct answer.

First execute block then constructor.

Question - 43

```
public class A {  
    static { System.out.println("static"); }  
    { System.out.println("block"); }  
    public A() {  
        System.out.println("A");  
    }  
  
    public static void main(String[] args) {  
        A a = new A();  
    }  
    What will be output ?
```

1. A block static
2. static block A

3.static A

4.A

Explanation :

B is the correct answer.

First execute static block, then block then constructor.

Question - 44

```
class Bird {  
    int i;  
    boolean b;  
    float f;  
  
    public Bird() {  
        System.out.println(i);  
        System.out.println(b);  
        System.out.println(f);  
    }  
}
```

What is the output?

1.0 false 0.0

2.0 true 0.0

3.Not Compile

4.None of the above

Explanation :

A is the correct answer.

Jvm initialize default values; Static variables also;

Question - 45

```
public class Test {  
    public static void main(String[] args){  
        StringBuffer a = new StringBuffer("Hello");  
        StringBuffer b = a.append("World");  
        System.out.println(b);  
    }  
}
```

What is the output?

1.World

2.HelloWorld

3.World

4. None of the above

Explanation :

B is the correct answer.

append

Question - 46

```
class c1
{
    static{
        System.out.println("static");
    }
    public static void main(String a[])
    {
        System.out.println("main");
    }
}
```

1. prints static main
2. prints main
3. prints main static
4. compiletime error

Explanation :

A is the correct answer.

First execute the static block when the class is executed.

Question - 47

When a byte is added to a char, what is the type of the result?

1. byte
2. int
3. long
4. none of the above

Explanation :

B is the correct answer.

The result of all arithmetic performed with the binary operators (not the assignment operators) is an int, a long, a float, or a double. Here byte and char are promoted to int, so the result is an int.

Question - 48

Which of the following statements are true?

- 1) The automatic garbage collection of the JVM prevents programs from ever running out of memory
- 2) A program can suggest that garbage collection be performed but not force it
- 3) Garbage collection is platform independent
- 4) An object becomes eligible for garbage collection when all references denoting it are set to null.

- 1.1 and 4
- 2.2 and 4
- 3.2 and 3
- 4.3 and 4

Explanation :

B is the correct answer.

If a program keeps creating new references without any being discarded it may run out of memory. Unlike most aspects of Java garbage collection is platform dependent

Question - 49

Is the bellow import is correct?

```
import static java.lang.Math.*;
```

- 1.correct, static import allow
- 2.Incorrect, static import does not allow
- 3.Can't say
- 4.None of the above

Explanation :

A is the correct answer.

The static import construct allows unqualified access to static members without inheriting from the type containing the static members. `import static java.lang.Math.*;` can be use directly `double r = cos(PI * 3);` no need `Math.PI`

Question - 50

What is the output ?

```
package bean;
```

```

public class Abc {
    public static int index_val = 10;
}

package com;
import static bean.Abc.index_val;

public class Test1 {

    public static void main(String... args) {
        System.out.println(index_val);
    }
}

```

- 1.10
- 2.compile error, index_val not defined
- 3.Compile error at import static bean.Abc.index_val;
- 4.None of the error

Explanation :

A is the correct answer.

The static import construct allows unqualified access to static members without inheriting from the type containing the static members. J2SE 5.0 or later allows static import like `import static bean.Abc.index_val;` and can be use directly `System.out.println(index_val);`

Question - 51

What will happen when you attempt to compile and run the following code

```

public class Test extends Thread{
    public static void main(String argv[]){
        Test b = new Test();
        b.start();
    }
    public void run(){
        System.out.println("Running");
    }
}

```

- 1.Compilation and run but no output
- 2.Compilation and run with the output "Running"
- 3.Compile time error with complaint of no Thread target
- 4.Compile time error with complaint of no access to Thread package

Explanation :

B is the correct answer.

This is perfectly legitimate if useless sample of creating an instnace of a Thread and causing its run method to execute via a call to the start method. The Thread class is part of the core java.lang package and does not need any explicit import statement.

Question - 52

What will happen when you attempt to compile and run the following code?

```
public class B extends Thread{
public static void main(String argv[]){
B b = new B();
b.run();
}
public void start(){
for (int i = 0; i <10; i++){
System.out.println("Value of i = " + i);
}
}
}
```

- 1.A compile time error indicating that no run method is defined for the Thread class
- 2.A run time error indicating that no run method is defined for the Thread class
- 3.Clean compile and at run time the values 0 to 9 are printed out
- 4.Clean compile but no output at runtime

Explanation :

D is the correct answer.

This is a bit of a sneaky one as I have swapped around the names of the methods you need to define and call when running a thread. If the for loop were defined in a method called public void run() and the call in the main method had been to b.start() The list of values from 0 to 9 would have been output

Question - 53

What will happen when you attempt to compile and run the following code

```
class Base{
protected int i = 99;
}
public class Ab{
private int i=1;
public static void main(String argv[]){
Ab a = new Ab();
a.hallow();
}

    abstract void hallow(){
        System.out.println("Claines "+i);
    }

}
```

- 1.Compile time error

- 2.Compilation and output of Claines 99
- 3.Compilation and output of Claines 1
- 4.Compilation and not output at runtime

Explanation :

A is the correct answer.

Abstract and native methods can't have a body: void hallow() abstract void hallow()

Question - 54

What is the output?

```
import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        Set s = new TreeSet();
        s.add("7");
        s.add(9);
        Iterator itr = s.iterator();
        while (itr.hasNext())
            System.out.print(itr.next() + " ");

    }
}
```

- 1.Compile error
- 2.Runtime Exception
- 3.7 9
- 4.None of the above

Explanation :

B is the correct answer.

□ Without generics, the compiler does not know what type is appropriate for this TreeSet, so it allows everything to compile. But at runtime he TreeSet will try to sort the elements as they are added, and when it tries to compare an Integer with a String it will throw a ClassCastException.

□ Exception in thread "main" java.lang.ClassCastException: java.lang.String cannot be cast to java.lang.Integer.

Question - 55

What is the output for the bellow code ?

```
import java.util.EnumSet;
```

```

import static java.lang.System.out;

public class UsingEnumSet {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
        SUNDAY
    }

    public static void main(String[] args) {

        EnumSet<Day> allDays = EnumSet.range(Day.MONDAY, Day.SUNDAY);
        System.out.println("All days: " + allDays);

    }
}

```

- 1.All days: [MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY]
- 2.All days: [MONDAY, SUNDAY]
- 3.Compile with error
- 4.None of the above

Explanation :

A is the correct answer.

EnumSet.range(from, to) : Creates an enum set initially containing all of the elements in the range defined by the two specified endpoints

Question - 56

Is the bellow statement is true ?

The null reference CANNOT be stored in an enum set.

- 1.true
- 2.false
- 3.Can't say
- 4.None of the above

Explanation :

A is the correct answer.

java.util.EnumSet Enums can be used in a special purpose Set implementation (EnumSet) which provides better performance. All of the elements in an enum set MUST come from a single enum type. Enum sets are represented internally as bit vectors. The EnumSet class defines new methods and inherits the ones in the Set/Collection interfaces. NOTE, all methods are static except for the clone() method. All methods return an EnumSet. The null reference CANNOT be stored in an enum set.

Question - 57

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a{1,3}b?c*");  
        Matcher m = p.matcher("aaab");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

X? X, once or not at all X* X, zero or more times X+ X, one or more times X{n} X, exactly n times X{n,} X, at least n times X{n,m} X, at least n but not more than m times

Question - 58

Fill the gap:

```
1. class Outer {  
2.     class Inner{ }  
3. }  
4. class Test {  
5.     public static void main(String[] args) {  
6.         Outer o = new Outer();  
7.  
8.     }  
9. }
```

Which, inserted at line 7, creates an instance of Bar?

Use the following fragments zero or many times

Inner a = new o.Inner();

Outer.Inner a = o.new Inner();

Inner a = new Inner();

Question - 59

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        Calendar c = new Calendar();  
        System.out.println(c.getTimeInMillis());  
    }  
  
}
```

What is the output for the above code ?

- 1.Returns this Calendar's time value in milliseconds.
- 2.Compile error : Cannot instantiate the type Calendar
- 3.Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

Compile error : Cannot instantiate the type Calendar. In order to create a Calendar instance, you have to use one of the overloaded getInstance() static factory methods:
Calendar cal = Calendar.getInstance();

Question - 60

```
public class A {}  
  
public class B implements Serializable {  
    A a = new A();  
    public static void main(String... args){  
        B b = new B();  
        try{  
            FileOutputStream fs = new FileOutputStream("b.ser");  
            ObjectOutputStream os = new ObjectOutputStream(fs);  
            os.writeObject(b);  
            os.close();  
  
        }catch(Exception e){  
            e.printStackTrace();  
        }  
    }  
  
}
```

What is the output for the above code ?

- 1.Compilation Fail
- 2.java.io.NotSerializableException: Because class A is not Serializable.
- 3.No Exception

4. None of the above

Explanation :

B is the correct answer.

`java.io.NotSerializableException: A` Because class A is not Serializable.